Carlos Esteban Monge Barquero
SOFT-01 – Principios de Programación 1
Docente: Verónica Mora Lezcano

UNIVERSIDAD CENFOTEC

Avance 1

Proyecto Final: Misión Orbita

Septiembre 2026

Introducción

Este documento presenta el Avance 1 del proyecto final "Misión Órbita", correspondiente al curso SOFT-01 Principios de Programación 1. Se documentan cuatro ejercicios que abarcan estructuras secuenciales, condicionales simples, condicionales dobles y condiciones compuestas. Para cada ejercicio se incluye la tabla de variables, el diagrama de flujo, el código fuente en Python y la tabla de casos de prueba con los resultados calculados a mano.

Repositorio: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita

Ejercicio 1. Cálculo de recursos necesarios (secuencial)

Calcula el combustible, oxígeno, energía y provisiones necesarios para una misión espacial de ida y vuelta, a partir de la cantidad de tripulantes y los días estimados de viaje.

Tabla de variables

Nombre	Tipo	Descripción
nombre_mision	str	Nombre de la misión ingresado por el usuario.
cantidad_tripulantes	int	Cantidad de tripulantes de la misión.
dias_viaje	int	Días estimados para llegar al destino.
combustible_ida	int	Combustible necesario para el viaje de ida.
combustible_regreso	int	Combustible necesario para el viaje de regreso.
combustible_total	int	Combustible total requerido (ida + regreso + reserva).
oxigeno_requerido	int	Oxígeno total requerido para toda la tripulación.
energia_requerida	int	Energía total requerida para la misión.
provisiones_requeridas	int	Provisiones totales requeridas para la tripulación.
CONSUMO_COMBUSTIBLE_DIA	int (Constante)	Consumo de combustible por día de viaje = 8.
RESERVA_COMBUSTIBLE	int (Constante)	Reserva fija de combustible = 10.
CONSUMO_OXIGENO_TRIPULANTE_DIA	int (Constante)	Consumo de oxígeno por tripulante por día = 2.
OXIGENO_EMERGENCIA	int (Constante)	Oxígeno adicional de emergencia = 5.
CONSUMO_ENERGIA_DIA	int (Constante)	Consumo de energía por día = 5.
ENERGIA_EXPLORACION	int (Constante)	Energía adicional para exploración = 10.
CONSUMO_PROVISIONES_TRIPULANTE_DIA	int (Constante)	Consumo de provisiones por tripulante por día = 1.
PROVISIONES_ADICIONALES	int (Constante)	Provisiones adicionales = 3.
TRAMOS_VIAJE	int (Constante)	Cantidad de tramos del viaje (ida y regreso) = 2.

Diagrama de flujo

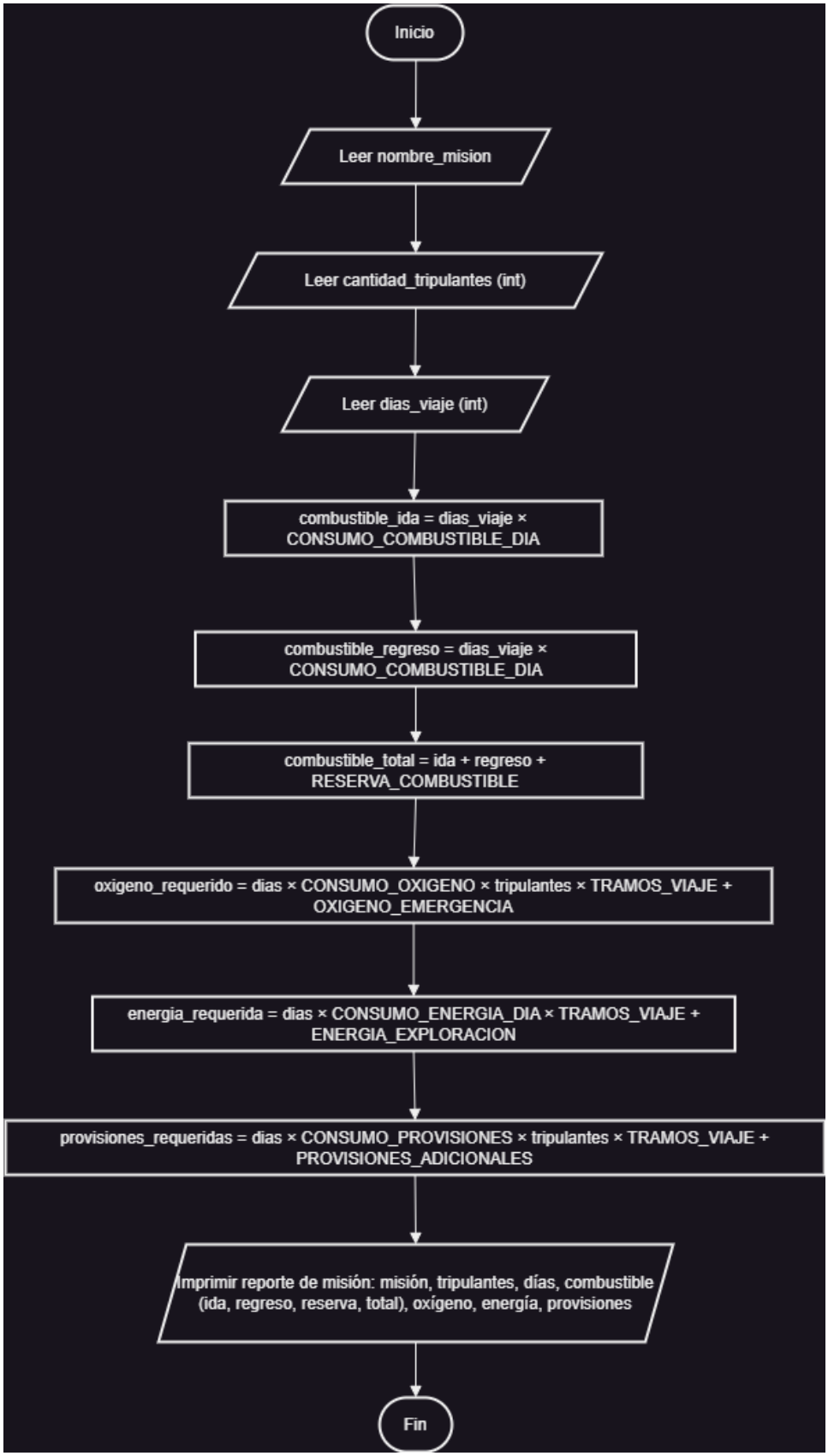


Ilustración 1 Puede verificarse en: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita/avance_1/ejercicio1_recursos.md

Descripción de los pasos del algoritmo:

1. Inicio.
2. Leer nombre_mision, cantidad_tripulantes y dias_viaje.
3. Calcular combustible_ida y combustible_regreso (dias_viaje × CONSUMO_COMBUSTIBLE_DIA).
4. Calcular combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE.
5. Calcular oxigeno_requerido, energia_requerida y provisiones_requeridas con sus fórmulas respectivas.
6. Mostrar todos los resultados por pantalla.
7. Fin.

Código fuente en Python

```
# Curso: SOFT-01 Principios de Programación 1 (SCV6)
# Archivo: ejercicio1_recursos.py
# Integrantes: Carlos Esteban Monge Barquero
# Fecha: 2026-10-03
# Versión: 1.0
# Descripción: Calcula los recursos (combustible, oxígeno, energía y
#              provisiones) necesarios para una misión espacial de ida y vuelta.
# Repo: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita

CONSUMO_COMBUSTIBLE_DIA = 8
RESERVA_COMBUSTIBLE = 10
CONSUMO_OXIGENO_TRIPULANTE_DIA = 2
OXIGENO_EMERGENCIA = 5
CONSUMO_ENERGIA_DIA = 5
ENERGIA_EXPLORACION = 10
CONSUMO_PROVISIONES_TRIPULANTE_DIA = 1
PROVISIONES_ADICIONALES = 3
TRAMOS_VIAJE = 2  # ida y regreso

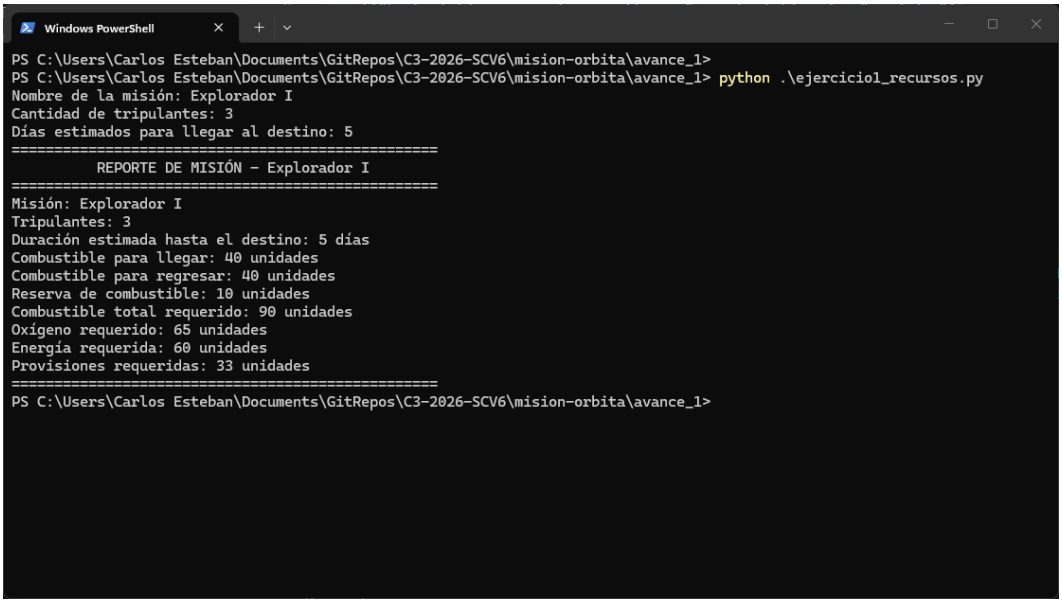
# Lectura de datos
nombre_mision = input("Nombre de la misión: ")
cantidad_tripulantes = int(input("Cantidad de tripulantes: "))
dias_viaje = int(input("Días estimados para llegar al destino: "))

# Cálculos
combustible_ida = dias_viaje * CONSUMO_COMBUSTIBLE_DIA
combustible_regreso = dias_viaje * CONSUMO_COMBUSTIBLE_DIA
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
oxigeno_requerido = (dias_viaje * CONSUMO_OXIGENO_TRIPULANTE_DIA * cantidad_tripulantes
                     * TRAMOS_VIAJE + OXIGENO_EMERGENCIA)
energia_requerida = (dias_viaje * CONSUMO_ENERGIA_DIA * TRAMOS_VIAJE
                     + ENERGIA_EXPLORACION)
provisiones_requeridas = (dias_viaje * CONSUMO_PROVISIONES_TRIPULANTE_DIA
                          * cantidad_tripulantes * TRAMOS_VIAJE
                          + PROVISIONES_ADICIONALES)

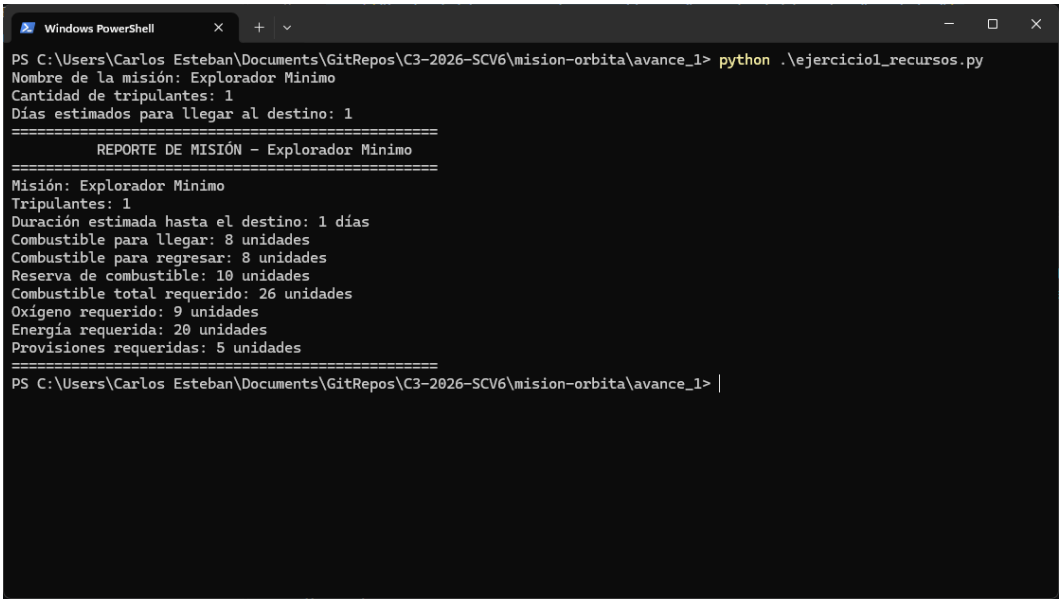
# Salida
print("=" * 50)
print("          REPORTE DE MISIÓN -", nombre_mision)
print("=" * 50)
print("Misión:", nombre_mision)
print("Tripulantes:", cantidad_tripulantes)
print("Duración estimada hasta el destino:", dias_viaje, "días")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Reserva de combustible:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Oxígeno requerido:", oxigeno_requerido, "unidades")
print("Energía requerida:", energia_requerida, "unidades")
print("Provisiones requeridas:", provisiones_requeridas, "unidades")
print("=" * 50)
```

Evidencia de ejecución

CP-01 — Explorador I (3 tripulantes, 5 días)



CP-02 — Explorador mínimo (1 tripulante, 1 día)



CP-03 — Misión grande (6 tripulantes, 10 días)

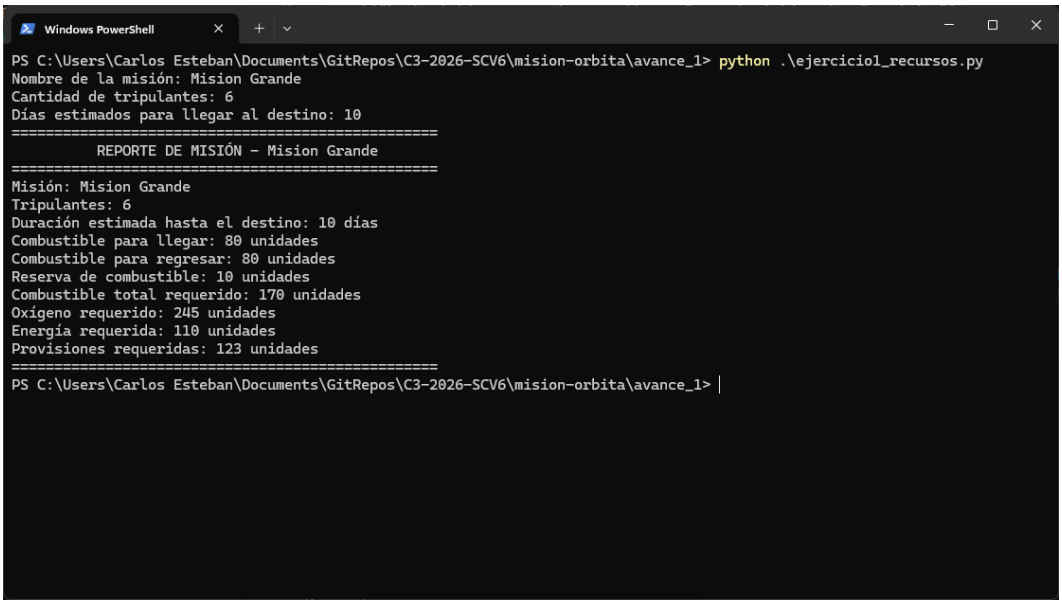


Tabla de casos de prueba

ID	Entradas	Resultado esperado	Resultado obtenido	Observación
CP-01	nombre_mision = "Explorador I" cantidad_tripulantes = 3 dias_viaje = 5	combustible_total = 90 oxigeno_requerido = 65 energia_requerida = 60 provisiones_requeridas = 33	combustible_total = 90 oxigeno_requerido = 65 energia_requerida = 60 provisiones_requeridas = 33 ✓ Coincide con esperado	Caso base del enunciado (Explorador I).
CP-02	nombre_mision = "Explorador mínimo" cantidad_tripulantes = 1 dias_viaje = 1	combustible_total = 26 oxigeno_requerido = 9 energia_requerida = 20 provisiones_requeridas = 5	combustible_total = 26 oxigeno_requerido = 9 energia_requerida = 20 provisiones_requeridas = 5 ✓ Coincide con esperado	Caso límite inferior: 1 tripulante, 1 día.
CP-03	nombre_mision = "Misión grande" cantidad_tripulantes = 6 dias_viaje = 10	combustible_total = 170 oxigeno_requerido = 245 energia_requerida = 110 provisiones_requeridas = 123	combustible_total = 170 oxigeno_requerido = 245 energia_requerida = 110 provisiones_requeridas = 123 ✓ Coincide con esperado	Caso con valores grandes de tripulantes y días.

Ejercicio 2. Verificación de combustible (condicional simple sin else)

Verifica si el combustible disponible alcanza para el viaje de ida y regreso más la reserva de emergencia, e informa cuando el margen adicional es bajo.

Tabla de variables

Nombre	Tipo	Descripción
combustible_disponible	int	Combustible disponible en la nave.
combustible_ida	int	Combustible requerido para el viaje de ida.
combustible_regreso	int	Combustible requerido para el viaje de regreso.
combustible_total	int	Combustible total requerido (ida + regreso + reserva).
combustible_adicional	int	Diferencia entre el combustible disponible y el requerido.
RESERVA_COMBUSTIBLE	int (Constante)	Reserva fija de combustible = 10.
MARGEN_MINIMO_ADICIONAL	int (Constante)	Margen mínimo adicional aceptable = 10.

Diagrama de flujo

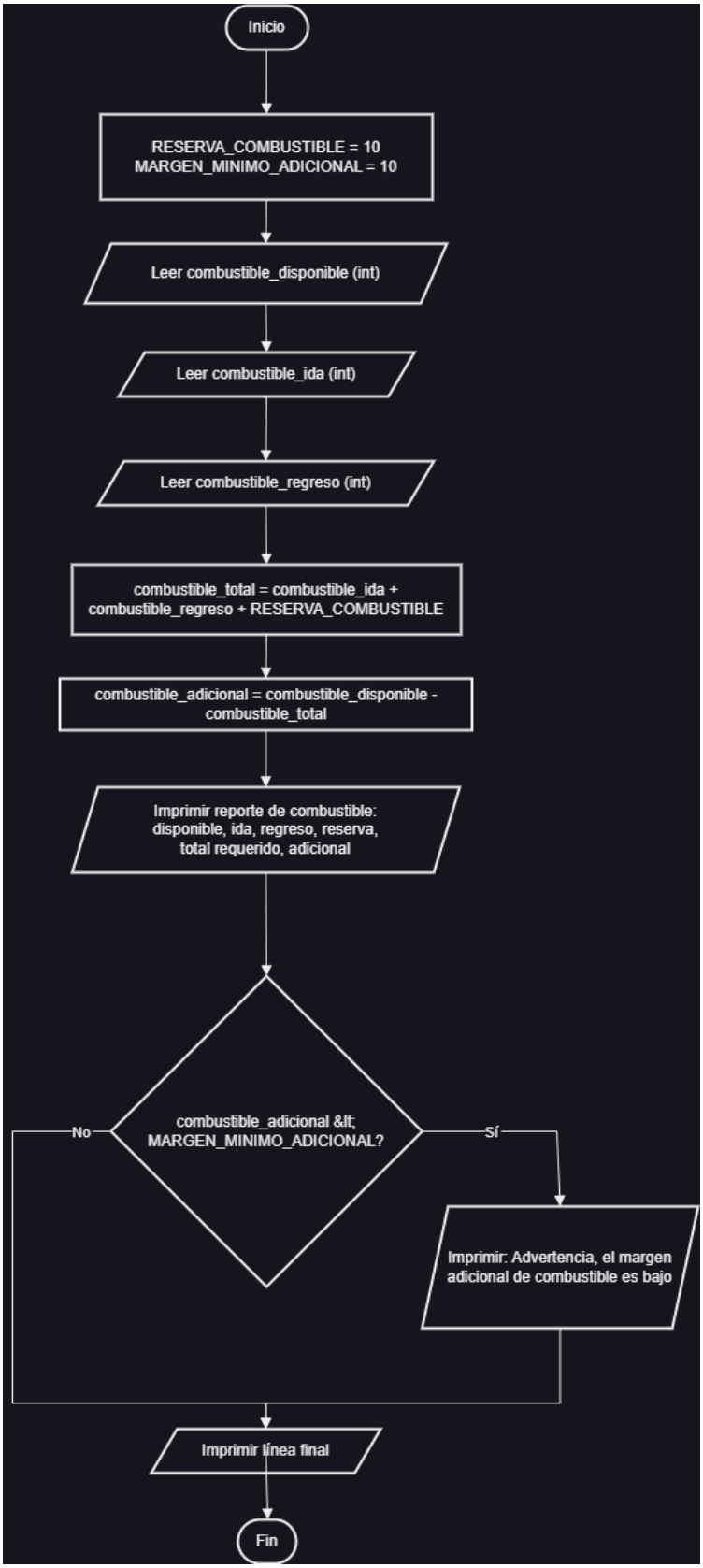


Ilustración 2 Puede verificarse en: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita/avance_1/ejercicio2_combustible.md

Descripción de los pasos del algoritmo

1. Inicio.
2. Leer combustible_disponible, combustible_ida y combustible_regreso.
3. Calcular combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE.
4. Calcular combustible_adicional = combustible_disponible - combustible_total.
5. Mostrar los datos de combustible calculados.
6. Si combustible_adicional < MARGEN_MINIMO_ADICIONAL, mostrar advertencia.
7. Fin.

Código fuente en Python

```
# Curso: SOFT-01 Principios de Programación 1 (SCV6)
# Archivo: ejercicio2_combustible.py
# Integrantes: Carlos Esteban Monge Barquero
# Fecha: 2026-10-04
# Versión: 1.0
# Descripción: Verifica si el combustible disponible alcanza para el viaje de
#              ida y regreso más la reserva de emergencia, e informa el margen.
# Repo: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita

RESERVA_COMBUSTIBLE = 10
MARGEN_MINIMO_ADICIONAL = 10

# Lectura de datos
combustible_disponible = int(input("Combustible disponible: "))
combustible_ida = int(input("Combustible requerido para el viaje de ida: "))
combustible_regreso = int(input("Combustible requerido para el viaje de regreso: "))

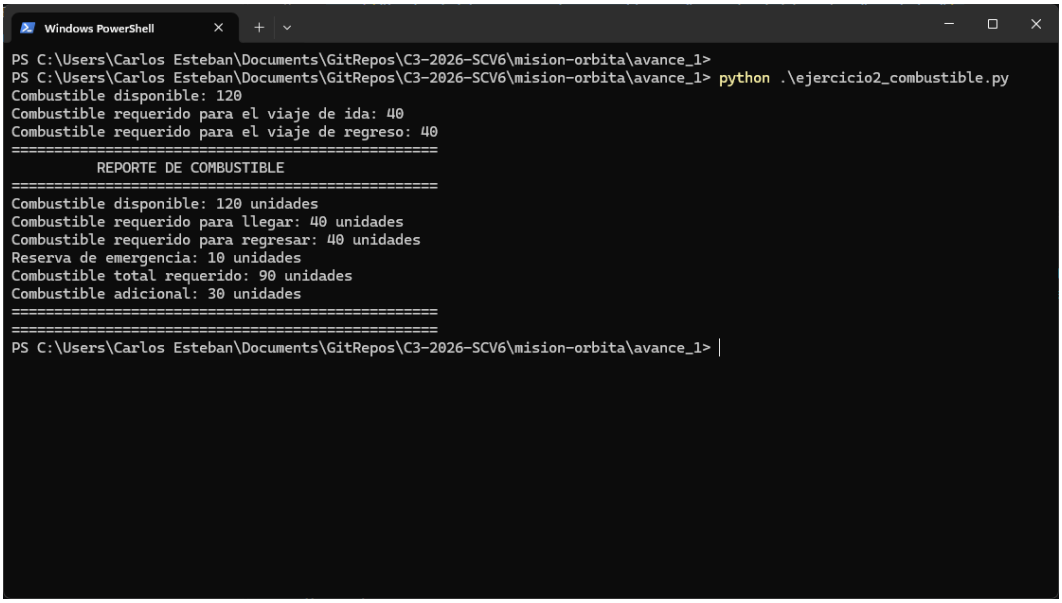
# Cálculos
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
combustible_adicional = combustible_disponible - combustible_total

# Salida
print("=" * 50)
print("          REPORTE DE COMBUSTIBLE")
print("=" * 50)
print("Combustible disponible:", combustible_disponible, "unidades")
print("Combustible requerido para llegar:", combustible_ida, "unidades")
print("Combustible requerido para regresar:", combustible_regreso, "unidades")
print("Reserva de emergencia:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Combustible adicional:", combustible_adicional, "unidades")

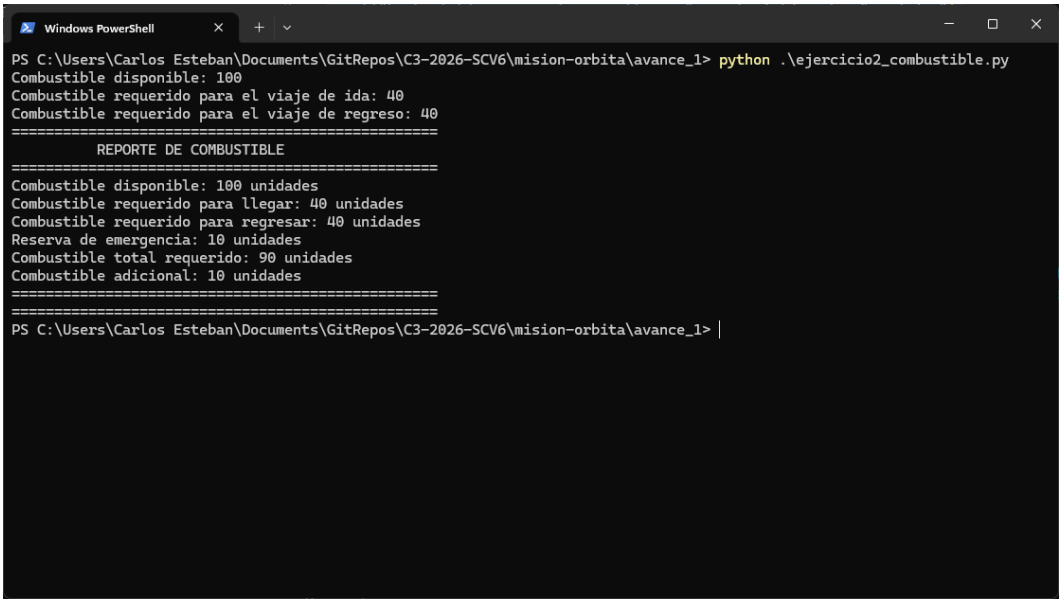
if combustible_adicional < MARGEN_MINIMO_ADICIONAL:
    print("Advertencia: el margen adicional de combustible es bajo.")
print("=" * 50)
```

Evidencia de ejecución

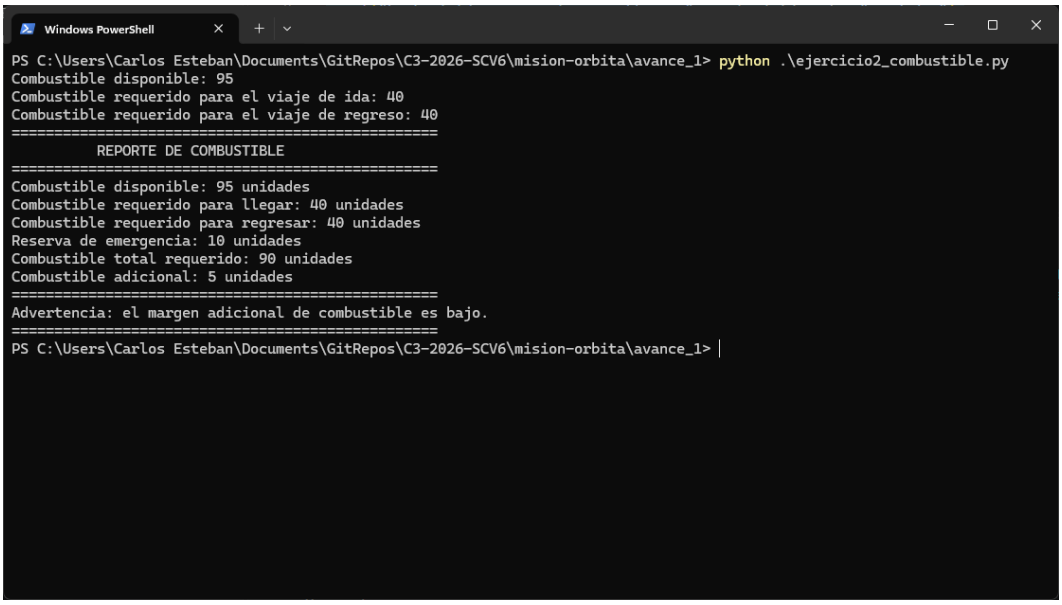
CP-04 — Margen amplio



CP-05 — Límite exacto del margen



CP-06 — Margen bajo (advertencia)



CP-07 — Combustible insuficiente (advertencia)

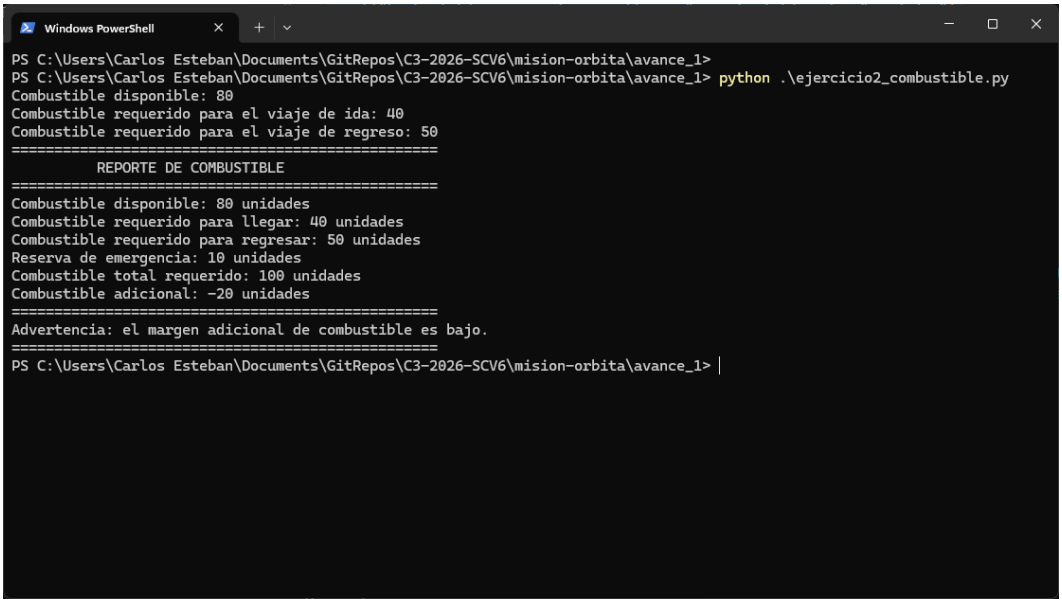


Tabla de casos de prueba

ID	Entradas	Resultado esperado	Resultado obtenido	Observación
CP-04	combustible_disponible = 120 combustible_ida = 40 combustible_regreso = 40	combustible_total = 90 combustible_adicional = 30 No se muestra advertencia	combustible_total = 90 combustible_adicional = 30 Sin advertencia ✓ Coincide con esperado	Margen amplio, sin advertencia.
CP-05	combustible_disponible = 100 combustible_ida = 40 combustible_regreso = 40	combustible_total = 90 combustible_adicional = 10 No se muestra advertencia (límite)	combustible_total = 90 combustible_adicional = 10 Sin advertencia (el límite exacto no activa la advertencia porque la condición es <) ✓ Coincide con esperado	Caso límite: adicional exactamente igual a 10 (no es < 10).
CP-06	combustible_disponible = 95 combustible_ida = 40 combustible_regreso = 40	combustible_total = 90 combustible_adicional = 5 Se muestra advertencia	combustible_total = 90 combustible_adicional = 5 Advertencia: el margen adicional de combustible es bajo. ✓ Coincide con esperado	Margen bajo, por debajo de 10.
CP-07	combustible_disponible = 80 combustible_ida = 40 combustible_regreso = 50	combustible_total = 100 combustible_adicional = -20 Se muestra advertencia	combustible_total = 100 combustible_adicional = -20 Advertencia: el margen adicional de combustible es bajo. ✓ Coincide con esperado	Combustible insuficiente (adicional negativo).

Ejercicio 3. Soporte a la tripulación (condicional doble)

Determina si la nave posee oxígeno y provisiones suficientes para sostener a la tripulación durante la misión, utilizando una estructura if-else.

Tabla de variables

Nombre	Tipo	Descripción
oxigeno_disponible	int	Oxígeno disponible en la nave.
oxigeno_requerido	int	Oxígeno requerido para la tripulación.
provisiones_disponibles	int	Provisiones disponibles en la nave.
provisiones_requeridas	int	Provisiones requeridas para la tripulación.

Diagrama de flujo

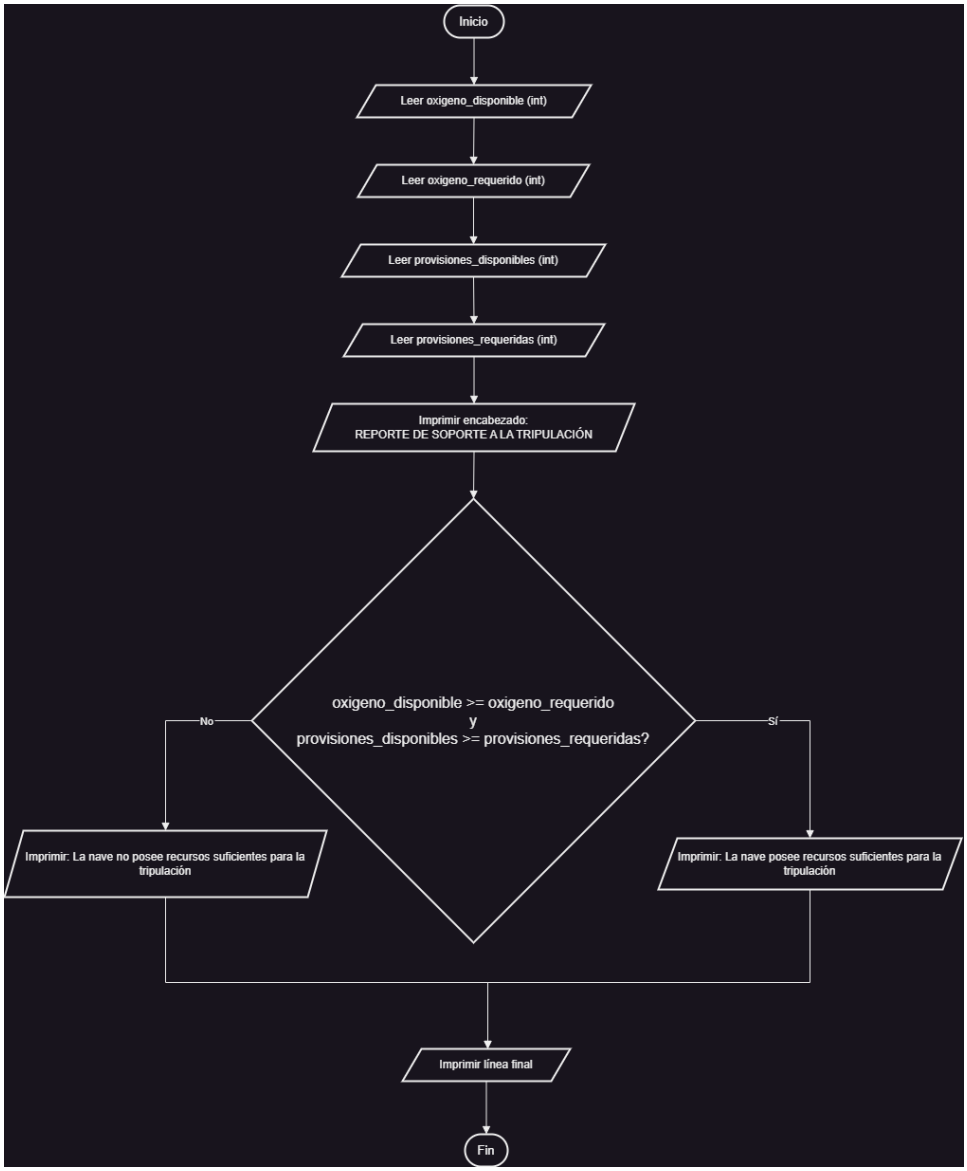


Ilustración 3 Puede verificarse en: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita/avance_1/ejercicio3_soporte_tripulacion.md

Descripción de los pasos del algoritmo

1. Inicio.
2. Leer oxigeno_disponible, oxigeno_requerido, provisiones_disponibles y provisiones_requeridas.
3. Si oxigeno_disponible >= oxigeno_requerido Y provisiones_disponibles >= provisiones_requeridas:
4. Mostrar "La nave posee recursos suficientes para la tripulación."
5. En caso contrario:
6. Mostrar "La nave no posee recursos suficientes para la tripulación."
7. Fin.

Código fuente en Python

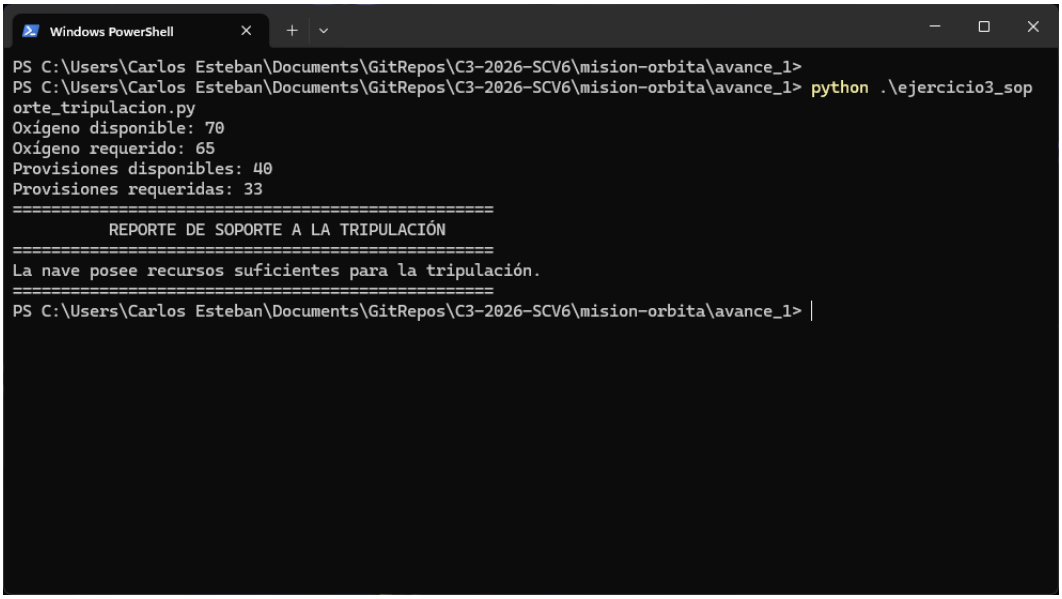
```
# Curso: SOFT-01 Principios de Programación 1 (SCV6)
# Archivo: ejercicio3_soporte_tripulacion.py
# Integrantes: Carlos Esteban Monge Barquero
# Fecha: 2026-10-05
# Versión: 1.0
# Descripción: Determina si la nave posee oxígeno y provisiones suficientes
#              para sostener a la tripulación durante la misión.
# Repo: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita

# Lectura de datos
oxigeno_disponible = int(input("Oxígeno disponible: "))
oxigeno_requerido = int(input("Oxígeno requerido: "))
provisiones_disponibles = int(input("Provisiones disponibles: "))
provisiones_requeridas = int(input("Provisiones requeridas: "))

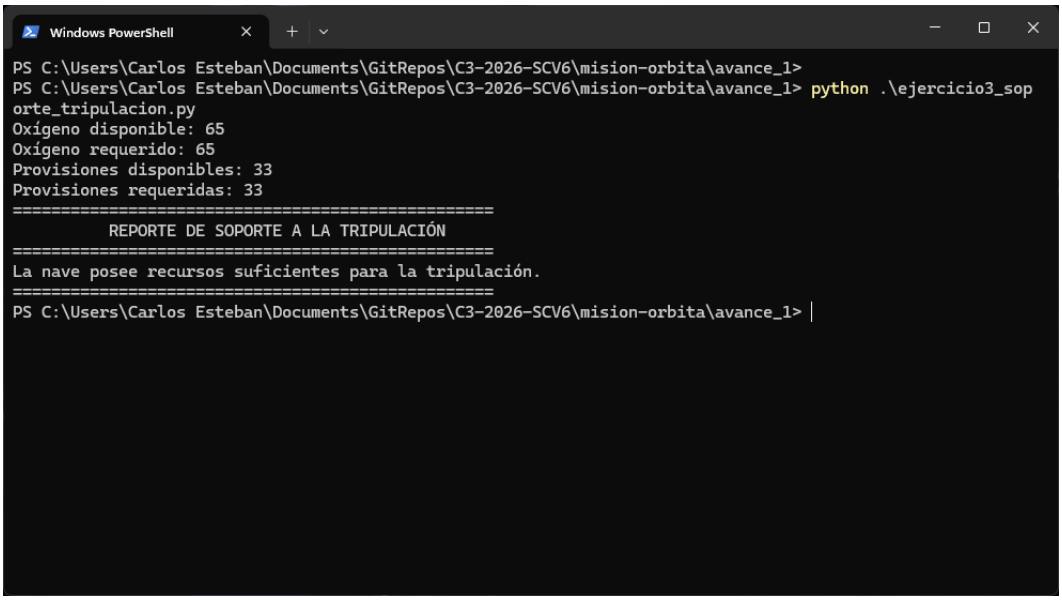
# Cálculos / validación
print("=" * 50)
print("          REPORTE DE SOPORTE A LA TRIPULACIÓN")
print("=" * 50)
if oxigeno_disponible >= oxigeno_requerido and provisiones_disponibles >=
provisiones_requeridas:
    # Salida
    print("La nave posee recursos suficientes para la tripulación.")
else:
    # Salida
    print("La nave no posee recursos suficientes para la tripulación.")
print("=" * 50)
```

Evidencia de ejecución

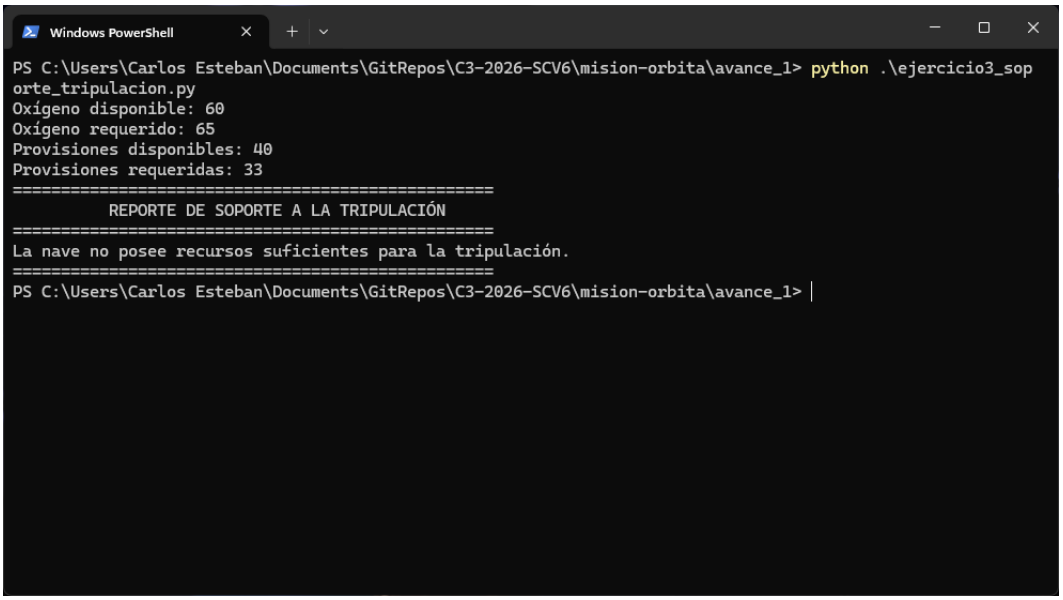
CP-08 — Ambos recursos suficientes



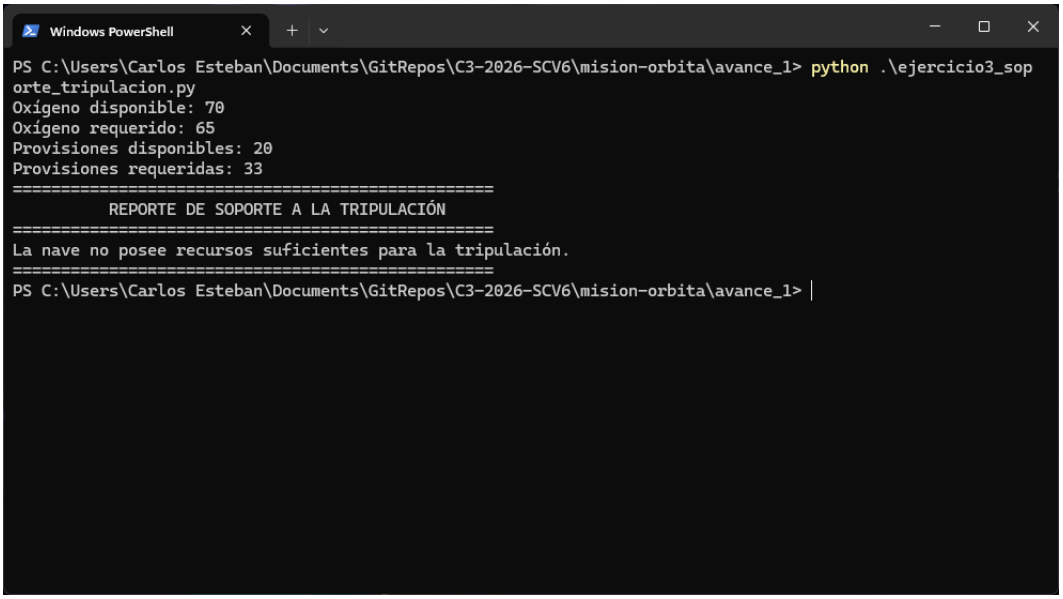
CP-09 — Límite exacto (ambos iguales)



CP-10 — Oxígeno insuficiente



CP-11 — Provisiones insuficientes



CP-12 — Ambos insuficientes

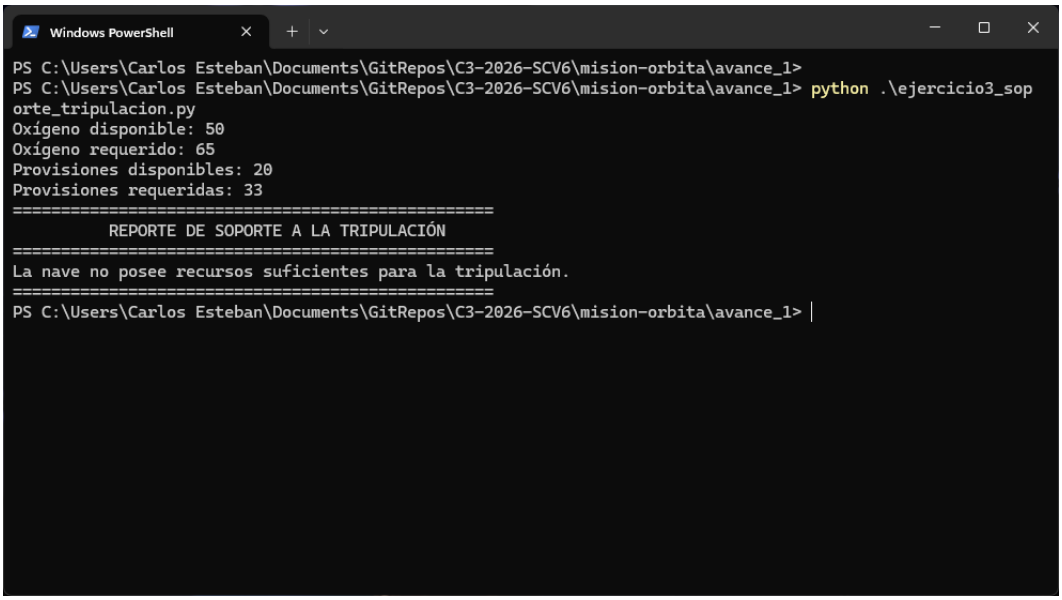


Tabla de casos de prueba

ID	Entradas	Resultado esperado	Resultado obtenido	Observación
CP-08	oxigeno_disponible = 70 oxigeno_requerido = 65 provisiones_disponibles = 40 provisiones_requeridas = 33	"La nave posee recursos suficientes para la tripulación."	"La nave posee recursos suficientes para la tripulación." ✓ Coincide con esperado	Ambos recursos suficientes.
CP-09	oxigeno_disponible = 65 oxigeno_requerido = 65 provisiones_disponibles = 33 provisiones_requeridas = 33	"La nave posee recursos suficientes para la tripulación."	"La nave posee recursos suficientes para la tripulación." ✓ Coincide con esperado	Caso límite: disponible igual a requerido en ambos recursos (>=).
CP-10	oxigeno_disponible = 60 oxigeno_requerido = 65 provisiones_disponibles = 40 provisiones_requeridas = 33	"La nave no posee recursos suficientes para la tripulación."	"La nave no posee recursos suficientes para la tripulación." ✓ Coincide con esperado	Oxígeno insuficiente, provisiones suficientes.
CP-11	oxigeno_disponible = 70 oxigeno_requerido = 65 provisiones_disponibles = 20 provisiones_requeridas = 33	"La nave no posee recursos suficientes para la tripulación."	"La nave no posee recursos suficientes para la tripulación." ✓ Coincide con esperado	Oxígeno suficiente, provisiones insuficientes.
CP-12	oxigeno_disponible = 50 oxigeno_requerido = 65 provisiones_disponibles = 20 provisiones_requeridas = 33	"La nave no posee recursos suficientes para la tripulación."	"La nave no posee recursos suficientes para la tripulación." ✓ Coincide con esperado	Ambos recursos insuficientes.

Ejercicio 4. Autorización de lanzamiento (condición compuesta)

Evalúa combustible, oxígeno, energía y provisiones disponibles frente a los requeridos para autorizar o no el lanzamiento, mediante una condición compuesta con múltiples operadores lógicos.

Tabla de variables

Nombre	Tipo	Descripción
combustible_disponible	int	Combustible disponible en la nave.
combustible_ida	int	Combustible requerido para llegar al destino.
combustible_regreso	int	Combustible requerido para regresar.
combustible_total	int	Combustible total requerido (ida + regreso + reserva).
oxigeno_disponible	int	Oxígeno disponible en la nave.
oxigeno_requerido	int	Oxígeno requerido para la misión.
energia_disponible	int	Energía disponible en la nave.
energia_requerida	int	Energía requerida para la misión.
provisiones_disponibles	int	Provisiones disponibles en la nave.
provisiones_requeridas	int	Provisiones requeridas para la misión.
RESERVA_COMBUSTIBLE	int (Constante)	Reserva fija de combustible = 10.

Diagrama de flujo

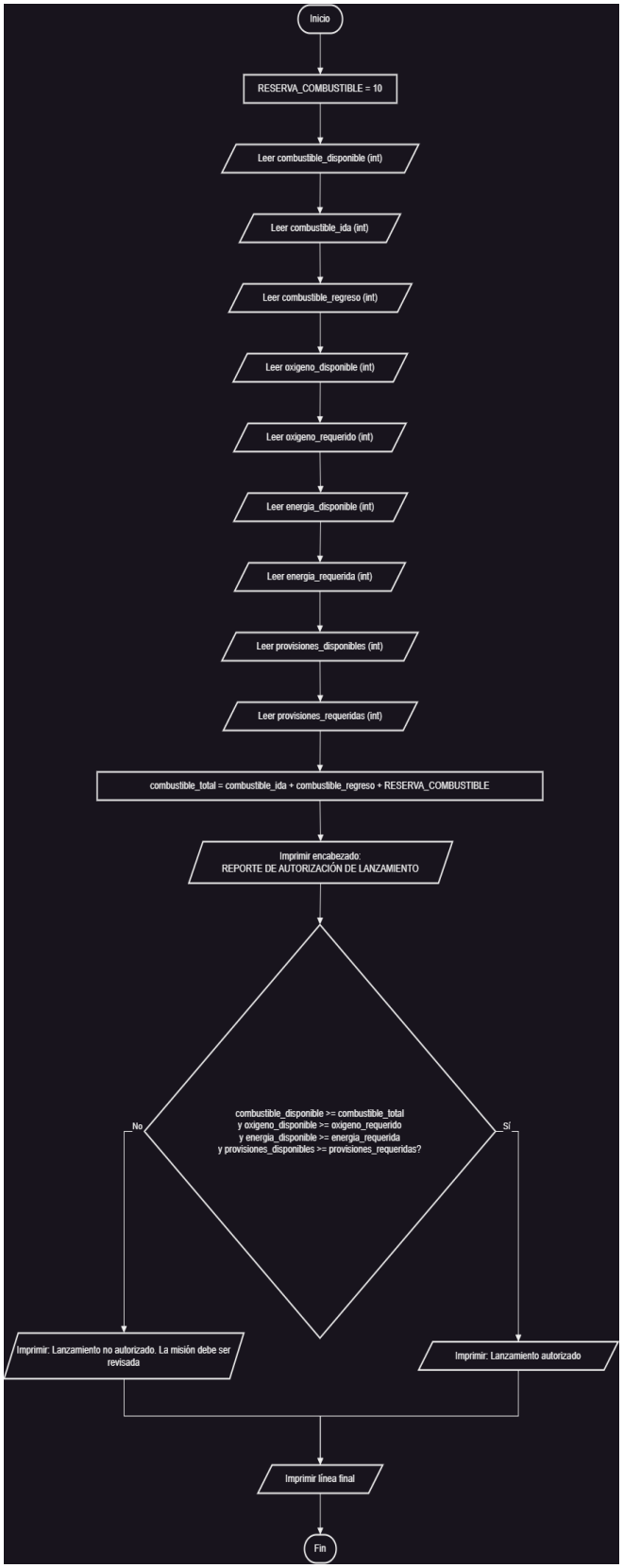


Ilustración 4 Puede verificarse en: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita/avance_1/ejercicio4_autorizacion.md

Descripción de los pasos del algoritmo

1. Inicio.
2. Leer combustible_disponible, combustible_ida, combustible_regreso, oxigeno_disponible,
3. oxigeno_requerido, energia_disponible, energia_requerida, provisiones_disponibles
4. y provisiones_requeridas.
5. Calcular combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE.
6. Si combustible_disponible >= combustible_total Y oxigeno_disponible >= oxigeno_requerido
7. Y energia_disponible >= energia_requerida Y provisiones_disponibles >= provisiones_requeridas:
8. Mostrar "Lanzamiento autorizado."
9. En caso contrario:
10. Mostrar "Lanzamiento no autorizado. La misión debe ser revisada."
11. Fin.

Código fuente en Python

```
# Curso: SOFT-01 Principios de Programación 1 (SCV6)
# Archivo: ejercicio4_autorizacion.py
# Integrantes: Carlos Esteban Monge Barquero
# Fecha: 2026-10-06
# Versión: 1.0
# Descripción: Evalúa combustible, oxígeno, energía y provisiones disponibles
#              frente a los requeridos para autorizar o no el lanzamiento.
# Repo: https://github.com/razzcr/C3-2026-SCV6/tree/main/mision-orbita

RESERVA_COMBUSTIBLE = 10

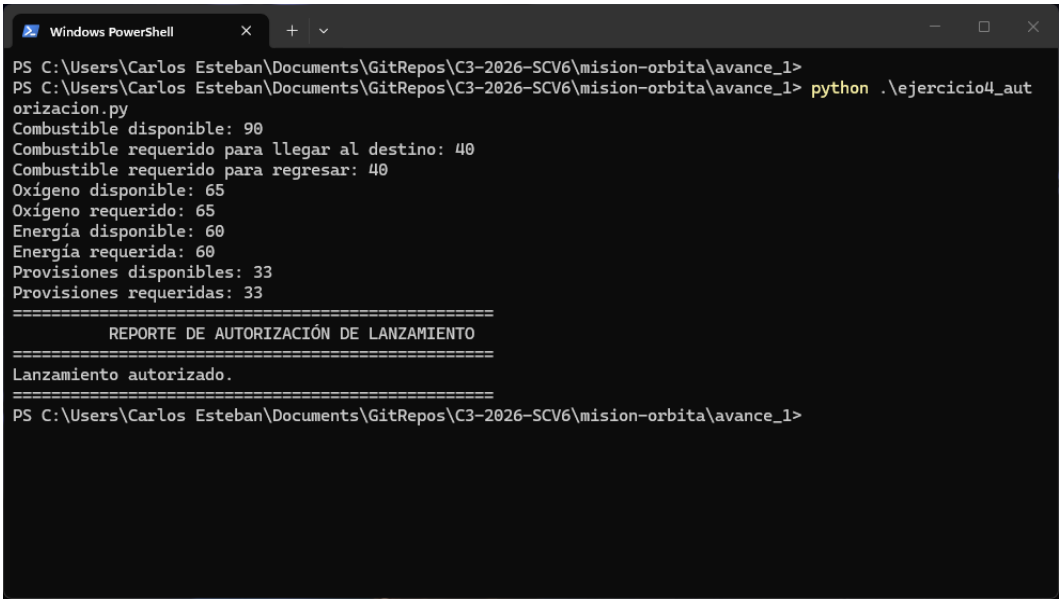
# Lectura de datos
combustible_disponible = int(input("Combustible disponible: "))
combustible_ida = int(input("Combustible requerido para llegar al destino: "))
combustible_regreso = int(input("Combustible requerido para regresar: "))
oxigeno_disponible = int(input("Oxígeno disponible: "))
oxigeno_requerido = int(input("Oxígeno requerido: "))
energia_disponible = int(input("Energía disponible: "))
energia_requerida = int(input("Energía requerida: "))
provisiones_disponibles = int(input("Provisiones disponibles: "))
provisiones_requeridas = int(input("Provisiones requeridas: "))

# Cálculos
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

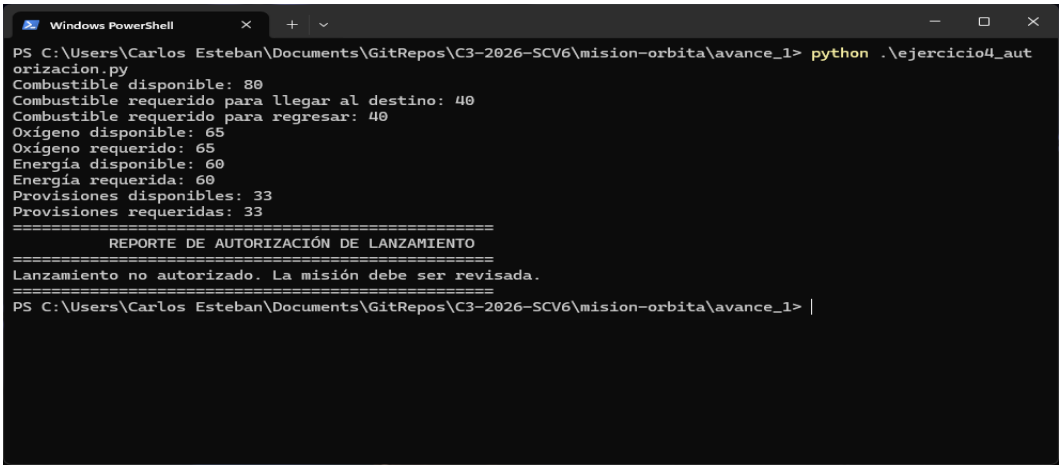
# Salida
print("=" * 50)
print("          REPORTE DE AUTORIZACIÓN DE LANZAMIENTO")
print("=" * 50)
if (combustible_disponible >= combustible_total
        and oxigeno_disponible >= oxigeno_requerido
        and energia_disponible >= energia_requerida
        and provisiones_disponibles >= provisiones_requeridas):
    print("Lanzamiento autorizado.")
else:
    print("Lanzamiento no autorizado. La misión debe ser revisada.")
print("=" * 50)
```

Evidencia de ejecución

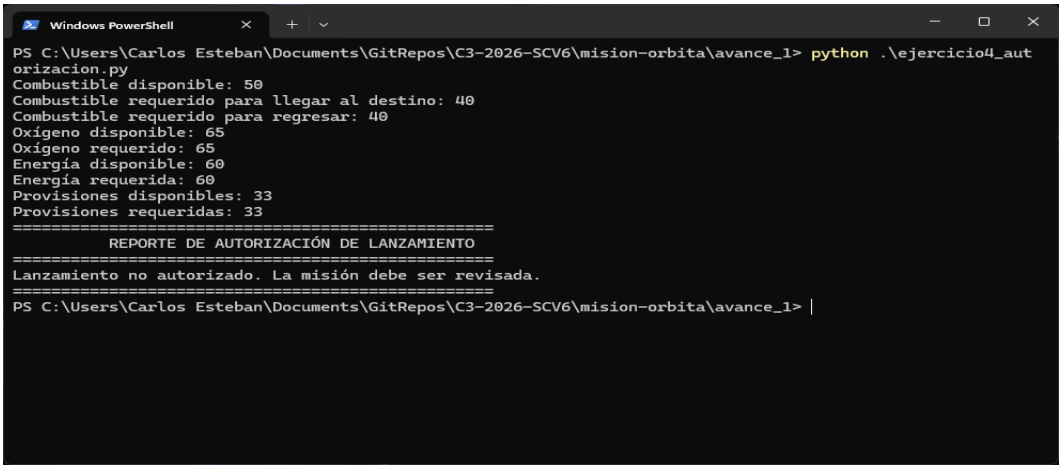
CP-13 — Límite exacto, todos los recursos suficientes (autorizado)



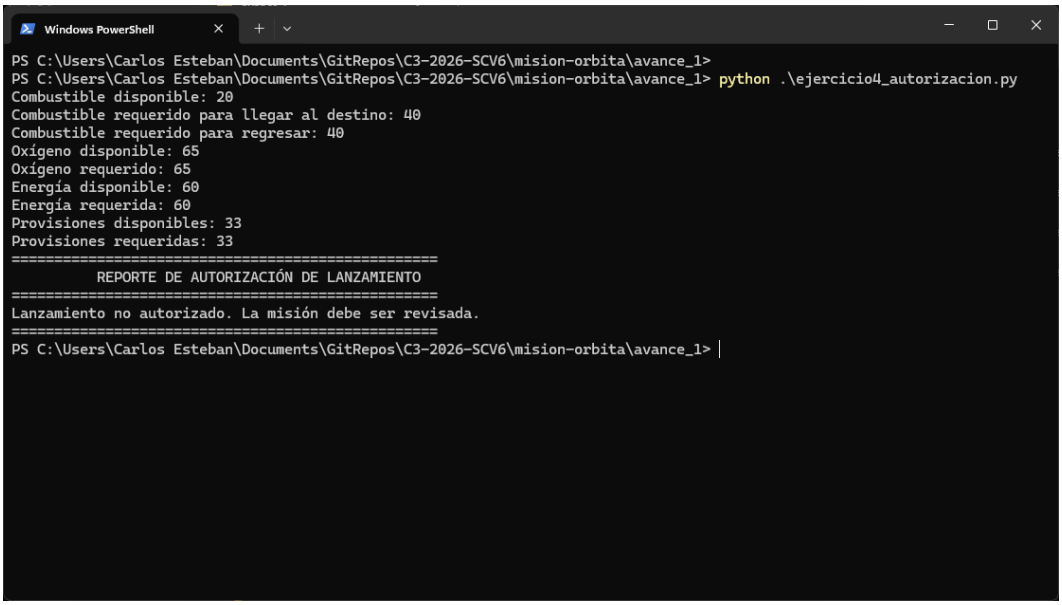
CP-14 — Combustible insuficiente (80 < 90)



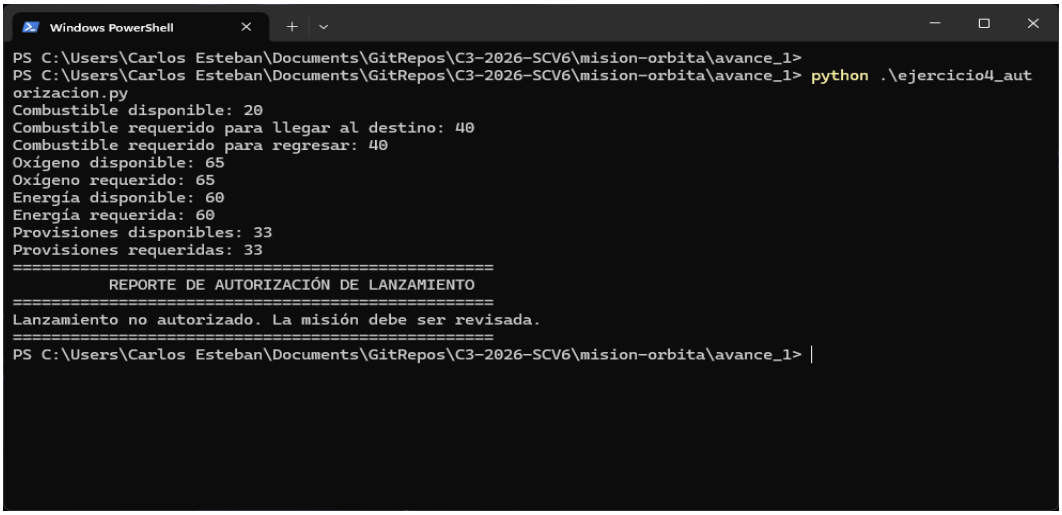
CP-15 — Combustible insuficiente (50 < 90)



CP-16 — Combustible muy insuficiente (20 < 90)



CP-17 — Combustible y oxígenos insuficientes



CP-18 — Todos los recursos insuficientes

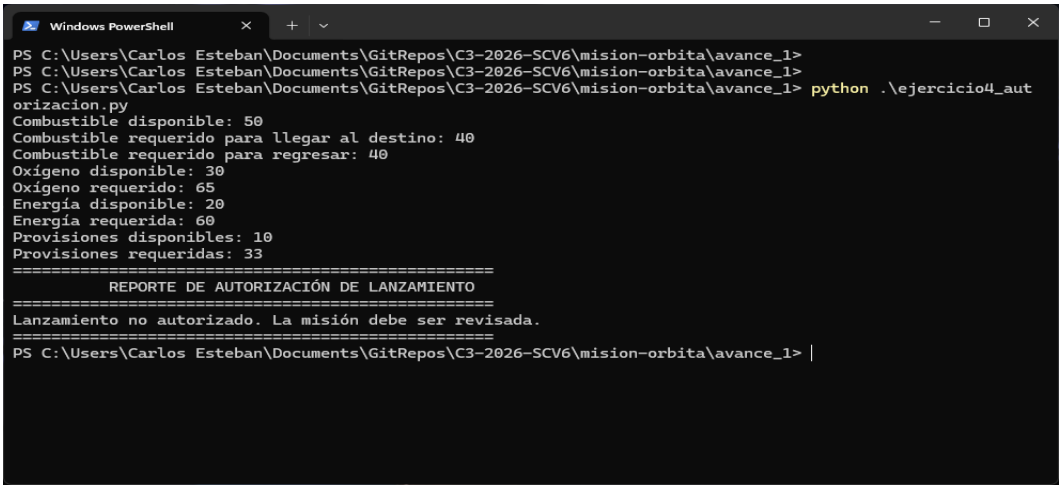


Tabla de casos de prueba

ID	Entradas	Resultado esperado	Resultado obtenido	Observación
CP-13	combustible_disponible = 90, combustible_ida = 40, combustible_regreso = 40 oxigeno_disponible = 65, oxigeno_requerido = 65 energia_disponible = 60, energia_requerida = 60 provisiones_disponibles = 33, provisiones_requeridas = 33	"Lanzamiento autorizado."	combustible_total = 90 (90/90) oxígeno 65/65, energía 60/60, provisiones 33/33 "Lanzamiento autorizado." ✓ Coincide con esperado	Todos los recursos exactamente en el límite (>=): se autoriza.
CP-14	combustible_disponible = 80 combustible_ida = 40, combustible_regreso = 40 oxigeno_disponible = 65, oxigeno_requerido = 65 energia_disponible = 60, energia_requerida = 60 provisiones_disponibles = 33, provisiones_requeridas = 33	"Lanzamiento no autorizado. La misión debe ser revisada."	combustible disponible 80 < total 90 oxígeno 65/65, energía 60/60, provisiones 33/33 "Lanzamiento no autorizado. La misión debe ser revisada." ✓ Coincide con esperado	Solo combustible insuficiente.
CP-15	combustible_disponible=50, combustible_ida=40, combustible_regreso=40, oxígeno_disponible=65, oxígeno_requerido=65, energía_disponible=60, energía_requerida=60, provisiones_disponibles=33, provisiones_requeridas=33	Combustible insuficiente (50 < 90 total requerido) “Lanzamiento no autorizado. La misión debe ser revisada.”	combustible disponible 50 < total 90 oxígeno 65/65, energía 60/60, provisiones 33/33 "Lanzamiento no autorizado. La misión debe ser revisada." ✓ Coincide con esperado	Combustible insuficiente (valor más bajo que CP-14).
CP-16	combustible_disponible = 20 combustible_ida = 40, combustible_regreso = 40 oxigeno_disponible = 65, oxigeno_requerido = 65 energia_disponible = 60, energia_requerida = 60 provisiones_disponibles = 33, provisiones_requeridas = 33	"Lanzamiento no autorizado. La misión debe ser revisada."	combustible disponible 20 < total 90 oxígeno 65/65, energía 60/60, provisiones 33/33 "Lanzamiento no autorizado. La misión debe ser revisada." ✓ Coincide con esperado	Combustible muy insuficiente.

CP-17	combustible_disponible=50, combustible_ida=40, combustible_regreso=40, oxígeno_disponible=30, oxígeno_requerido=65, energía_disponible=60, energía_requerida=60, provisiones_disponibles=33, provisiones_requeridas=33	Combustible y oxígenos insuficientes. “Lanzamiento no autorizado. La misión debe ser revisada.”	combustible disponible 50 < total 90 oxígeno disponible 30 < requerido 65 energía 60/60, provisiones 33/33 "Lanzamiento no autorizado. La misión debe ser revisada." ✓ Coincide con esperado	Combustible y oxígenos insuficientes.
CP-18	combustible_disponible = 50, oxigeno_disponible = 30, energia_disponible = 20, provisiones_disponibles = 10 (todos insuficientes)	"Lanzamiento no autorizado. La misión debe ser revisada."	combustible disponible 50 < total 90 oxígeno 30 < 65, energía 20 < 60, provisiones 10 < 33 "Lanzamiento no autorizado. La misión debe ser revisada." ✓ Coincide con esperado	Todos los recursos insuficientes.